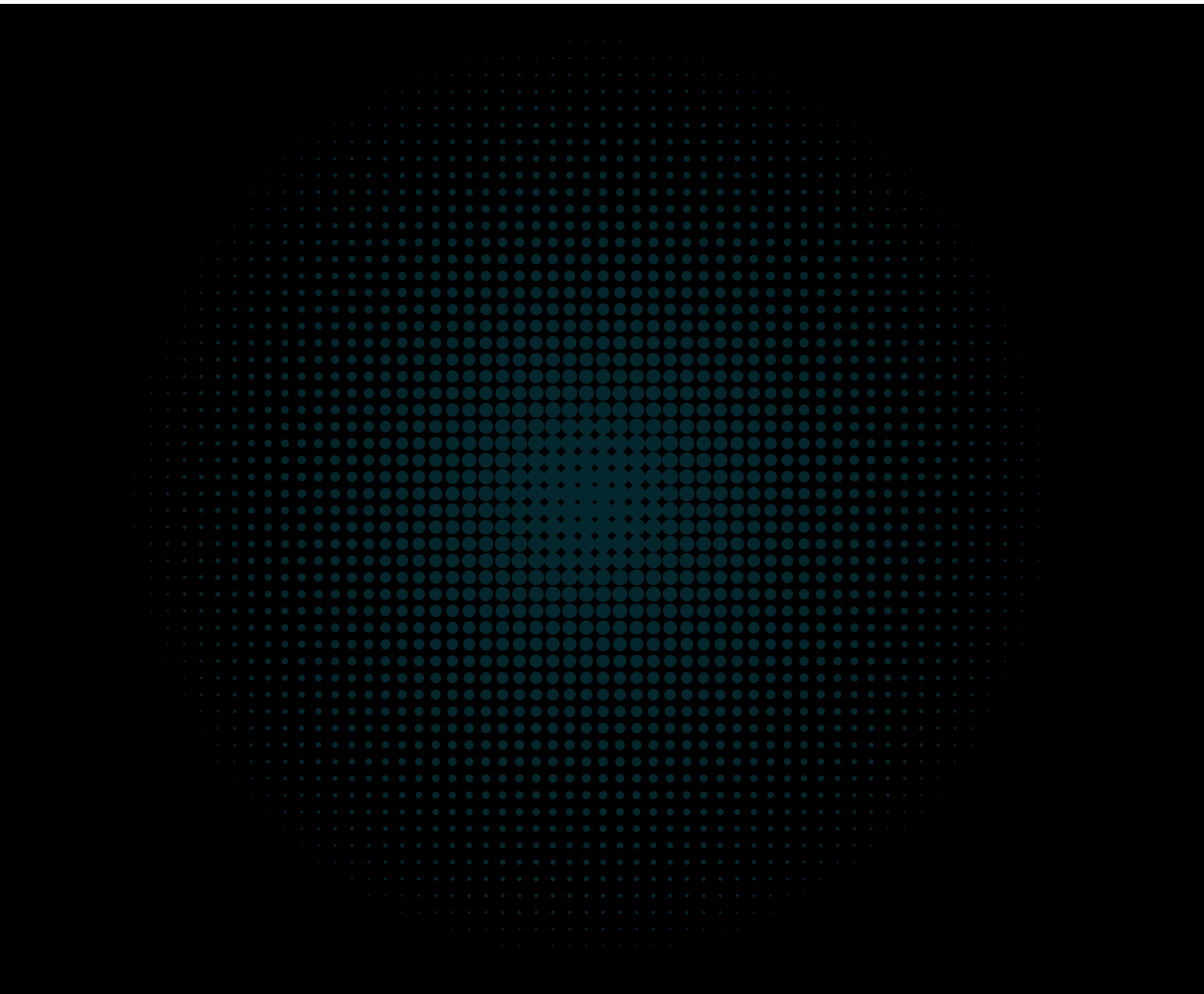
cmta.

RuleEngine Specification

Functional specifications of the RuleEngine for CMTAT and ERC-3643 tokens on Ethereum and EVM compatible blockchains.

RuleEngine version: v3.0.0-rc6
First published: August 2026



RuleEngine

This repository includes the RuleEngine contracts for CMTAT and ERC-3643 tokens.

The RuleEngine is an external contract used to apply transfer restrictions to another contract, such as CMTAT and ERC-3643 tokens. Acting as a controller, it can call different contract rules and apply these rules on each transfer.

> This project has not undergone an audit and is provided as-is without any warranties.

Table of Contents

- Contract Variants
- Motivation
- How it works
  - How to set it
  - Making `setCompliance` work with RuleEngine
- How to include it
  - Like CMTAT
  - Like ERC-3643
- Interface
  - CMTAT
  - ERC-3643
- Technical
  - Dependencies
  - Access Control
  - ERC-165 Support by Deployment Version
  - UML
  - Graph
- Functionality
  - Available Rules
  - Gasless support (ERC-2771)
  - Upgradeable
  - Urgency mechanism
- Ethereum API
  - Contract Constructors
  - RuleEngineBase
  - VersionModule
  - TokenBindingModule
  - ERC3643ComplianceModule
  - ERC3643ComplianceExtendedModule
  - RulesManagementModule
- Security
  - Vulnerability disclosure
  - Audit
  - Tools

- Documentation
- Toolchains and Usage
  - Configuration
  - Toolchain installation
  - Initialization
  - Compilation
  - Contract size
  - Testing
  - Coverage
  - Deployment
  - Solidity style guideline
- Intellectual property

Contract Variants

Three deployable contracts are available:

Contract	Access Control	Interface	Use Case
`RuleEngine`	Role-Based (AccessControlEnumerable)	RBAC roles	Multi-operator environments with granular permissions
`RuleEngineOwnable`	ERC-173 Ownership	`Ownable`	Single-owner setups, simpler administration
`RuleEngineOwnable2Step`	ERC-173 Ownership (two-step transfer)	`Ownable2Step`	Single-owner setups with safer ownership handover

ERC-3643 compliance specification indicates the use of ERC-173.

> The standard relies on ERC-173 to define contract ownership, with the owner having the responsibility of setting the Compliance parameters and binding the Compliance to a Token contract.

All deployable contracts share the same core functionality (`RuleEngineBase`, directly or through `RuleEngineOwnableShared`) and support:

- ERC-1404 transfer restrictions
- ERC-3643 compliance interface
- ERC-2771 meta-transactions (gasless)
- Multiple token bindings

> **Warning (shared engine across multiple tokens):** A "multi-tenant" setup here means one RuleEngine instance is shared by several token contracts (all bound through `bindToken`). In this setup, tokens must be equally trusted and governed together. ERC-3643 callbacks (`transferred`, `created`, `destroyed`) do not pass the token address to rules, so stateful/accounting rules are not safe for mutually untrusted tokens sharing the same engine.

Motivation

- Why use a dedicated contract with rules instead of implementing it directly in CMTAT or ERC-3643 tokens?

There are several reasons to do this:

- Flexibility: These different features are not standard and common to all tokens. From an implementation perspective, using a rule engine with custom rules allows for each issuer or contract user to decide which rules to apply.
- Code efficiency: The CMTAT token (and generally also all ERC-3643 tokens) is currently "heavy," meaning its contract code size is close to the maximum limit. This makes it challenging to add new features directly inside the token contract.
- Reusability:
    - The RuleEngine can be used inside other contracts besides CMTAT. For instance, the RuleEngine has been used in our contract to distribute dividends.
    - A same deployed `RuleEngine` can also be used with several different tokens if the rules allow it, which is the case for all read-only rules.

Why use this `RuleEngine` contract instead of setting the `rule` directly in the token contract?

- Using a RuleEngine allows to call several different rules. For example, a blacklist rule to allow the issuer to manage its own list of blacklisted addresses and a sanctionlist rule to use the Chainalysis oracle for sanctions screening to forbid transfers from addresses listed in sanctions designations published by organizations such as the US, EU, or UN.

When may the use of `RuleEngine` not be appropriate?

- If you plan to call only one rule (e.g a whitelist rule), it could make sense to directly set the rule in the token contract instead of using a RuleEngine. This will simplify configuration and reduce runtime gas costs.

How it works

This diagram illustrates how a transfer with a CMTAT or ERC-3643 token with a RuleEngine works:

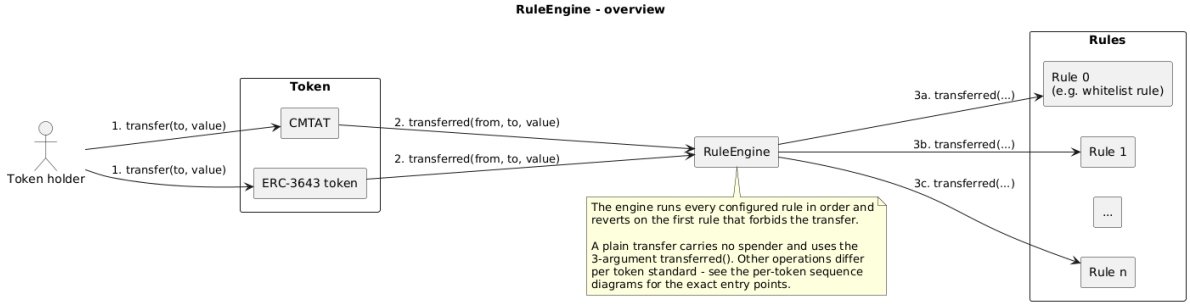


Diagram source: doc/schema/plantuml/ruleengine-overview.puml.

1. The token holders initiate a transfer transaction on the token contract.
2. The transfer function inside the token calls the ERC-3643 function `transferred` from the RuleEngine with the following parameters inside: `from, to, value`.
3. The Rule Engine calls each rule separately. If the transfer is not authorized by the rule, the rule must directly revert (no return value).

CMTAT and ERC-3643 tokens use **disjoint entry points** on the RuleEngine, so each has its own sequence diagram. The 4-argument `transferred(spender, from, to, value)` is declared by CMTAT's `IRuleEngine` and is never reached by an ERC-3643 token; conversely `created` / `destroyed` are declared by `IERC3643Compliance` and are never called by CMTAT. Only the 3-argument `transferred(from, to, value)` is shared.

With a CMTAT token

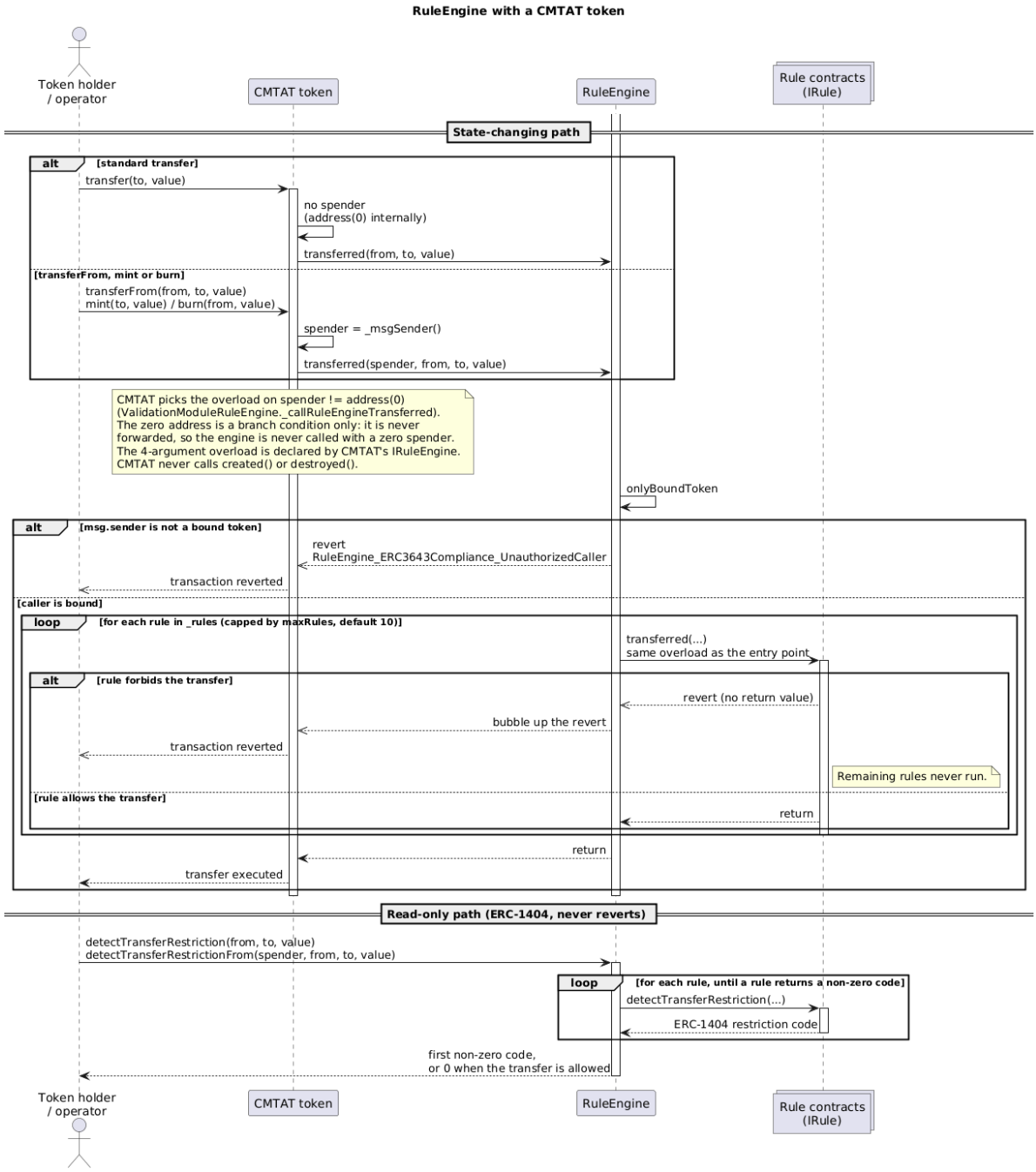


Diagram source: doc/schema/plantuml/ruleengine-flow-cmtat.puml.

With an ERC-3643 token

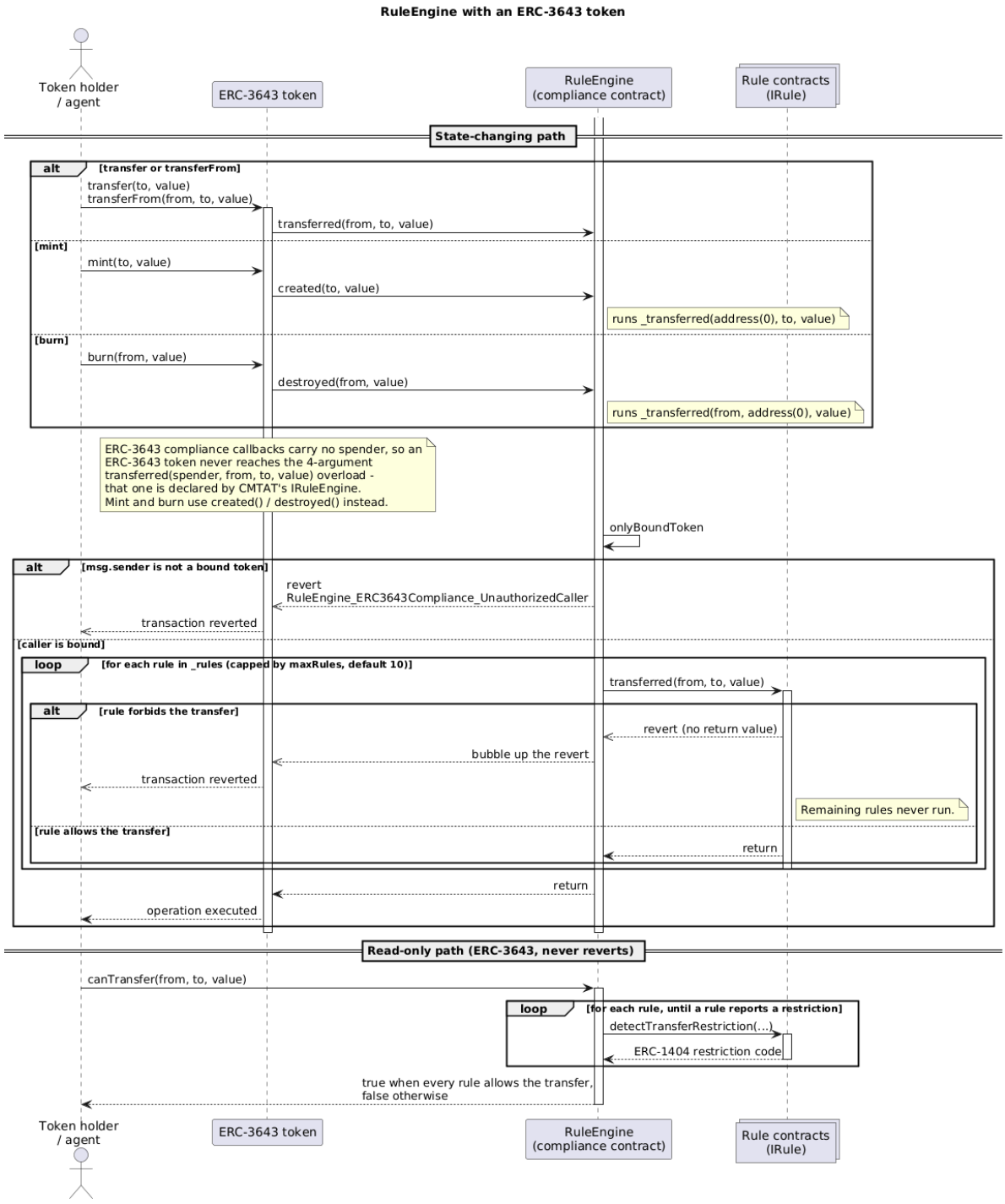


Diagram source: doc/schema/plantuml/ruleengine-flow-erc3643.puml.

> **Warning:** The RuleEngine iterates over all configured rules on every transfer (and on every call to `detectTransferRestriction`, `canTransfer`, etc.). Adding a large number of rules increases gas consumption for each transfer and may eventually exceed the block gas limit, effectively preventing any transfer from succeeding. An on-chain rule cap is enforced (`maxRules`), set to `10` by default, and can be changed by governance (`DEFAULT_ADMIN_ROLE` on `RuleEngine`, owner on ownable variants). A misconfigured or gas-heavy rule can still impact all transfers.

> **Warning (restriction code conventions):** Rule implementations should use unique ERC-1404 restriction codes across the rule set. If several rules intentionally share the same restriction code, they should return the exact same `messageForTransferRestriction` text for that code to avoid inconsistent operator/user feedback.

How to set it

Compatibility

RuleEngine version	Compatible Versions
v3.0.0-rc4	CMTAT ≥ v3.0.0 CMTAT target version: v3.3.0-rc1
v3.0.0-rc3	CMTAT ≥ v3.0.0 CMTAT target version: v3.3.0
v3.0.0-rc2	CMTAT ≥ v3.0.0 CMTAT target version: v3.2.0
v1.0.2.1	CMTAT v2.3.0 (audited)

CMTAT v3.0.0

CMTAT provides the following function to set a RuleEngine inside a CMTAT token:

```
setRuleEngine(IRuleEngine ruleEngine_)
```

This function is defined in the extension module `ValidationModuleRuleEngine`

ERC-3643 token

ERC-3643 defined the following function in the standard interface to set a compliance contract

```
setCompliance(address _compliance)
```

Making `setCompliance` work with RuleEngine

RuleEngine supports the ERC-3643/T-REX pattern where the token contract binds and unbinds itself when `setCompliance` is called.

In other words, a token can call:

- `bindToken(address(this))`
- `unbindToken(address(this))`

To keep this feature secure, self-bind/self-unbind is gated:

- A token can call `bindToken(address(this))` and `unbindToken(address(this))` only if it was explicitly approved first.
- Approval is set by governance/compliance admin using:
  - `setTokenSelfBindingApproval(address token, bool approved)`
- Approval status can be checked with:
  - `isTokenSelfBindingApproved(address token)`

This preserves compatibility with ERC-3643 tokens that do:

```
if (address(_tokenCompliance) != address(0)) {
    _tokenCompliance.unbindToken(address(this));
}
_tokenCompliance = IModularCompliance(_compliance);
_tokenCompliance.bindToken(address(this));
```

while preventing arbitrary third-party contracts from self-binding.

Recommended operational sequence:

1. On the target RuleEngine, grant self-binding approval for the token.
2. Call token `setCompliance(newRuleEngine)`.
3. (Optional) Revoke self-binding approval after migration if no longer needed.

How to include it

While the RuleEngine has been designed for CMTAT and ERC-3643 tokens, it can be used with other contracts to apply transfer restrictions.

For that, the only thing to do is to import in your contract the interface `IRuleEngine`(CMTAT) or `IERC3643Compliance` (ERC-3643), which declares the corresponding functions to call by the token contract. This interface can be found here.
If you need non-standard helper functions (batch bind/unbind, self-binding approval APIs, multi-token getter), use `IERC3643ComplianceExtended`.

Like CMTAT

Before each ERC-20 transfer, mint, or burn, CMTAT calls the RuleEngine through the internal function `_checkTransferred`, which dispatches to one of two `transferred` overloads depending on whether a non-zero spender is present.

```
// Called when spender == address(0)
function transferred(address from, address to, uint256 value)

// Called when spender != address(0)
function transferred(address spender, address from, address to, uint256 value)
```

CMTAT v3.3.0 — mint and burn use the spender path

Since CMTAT v3.3.0, **mint and burn operations also go through the 4-argument overload**, with the operator (minter or burner) passed as `spender`:

Operation	`spender`	`from`	`to`
`transfer` / `transferFrom`	caller / approved spender	token holder	recipient
`mint`	minter (`_msgSender()`)	`address(0)`	recipient
`burn`	burner (`_msgSender()`)	token holder	`address(0)`

The 3-argument overload is only called when `spender == address(0)`, which does not occur in normal CMTAT v3.3.0 flows.

> **Rule authoring note:** Rules that check the `spender` argument in `transferred(spender, from, to, value)` must explicitly handle the mint (`from == address(0)`) and burn (`to == address(0)`) cases. A spender check that is intended only for `transferFrom` will also fire for mints and burns unless the rule skips it when `from` or `to` is the zero address. See `RuleWhitelist` and `RuleSpenderWhitelist` in the Rules repository for reference implementations.

For example, CMTAT defines the interaction with the RuleEngine inside a specific module, ValidationModuleRuleEngine and CMTATBaseRuleEngine.

- ValidationModuleRuleEngine

```
/* ============ State functions ============ */
function _transferred(address spender, address from, address to, uint256 value) internal virtual returns (bool){
    if(!_canTransferGenericByModule(spender, from, to)){
        return false;
    } else {
         IRuleEngine ruleEngine_ = ruleEngine();
         if (address(ruleEngine_) != address(0)){
             if(spender != address(0)){
                ruleEngine_.transferred(spender, from, to, value);
             } else {
                ruleEngine_.transferred(from, to, value);
             }
        }
    }
    return true;
}
```

- CMTATBaseRuleEngine

```
function _checkTransferred(address spender, address from, address to, uint256 value) internal virtual override(CMTATBaseCommon) {
    CMTATBaseCommon._checkTransferred(spender, from, to, value);
    require(ValidationModuleRuleEngine._transferred(spender, from, to, value), Errors.CMTAT_InvalidTransfer(from, to, value));
}
```

This function `_transferred` is called before each transfer/burn/mint through the internal function `_checkTransferred`.

Like ERC-3643

The ERC-3643 defines several functions used as entrypoints for an ERC-3643 token.

As for CMTAT, the main entrypoint is `transferred` which must be called for each ERC-20 transfer.

Contrary to CMTAT, ERC-3643 does not apply restriction on the spender address (`transferFrom`).

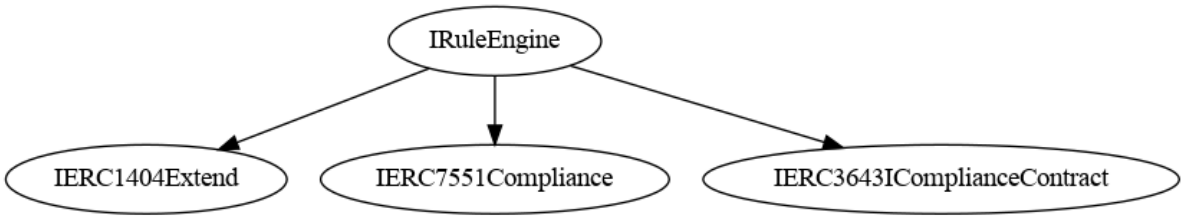
They are the following:

```
// read-only function
function canTransfer(address from, address to, uint256 value) external view
returns (bool);
// ERC-20 transfer
function transferred(address from, address to, uint256 value) external;
// mint
function created(address to, uint256 value) external;
// burn
function destroyed(address from, uint256 value) external;
```

Interface

CMTAT

The `RuleEngine` base interface is defined in the CMTAT repository.



It inherits from several others interfaces: `IERC1404`, `IERC1404Extend`, `IERC7551Compliance`, `IERC3643ComplianceContract`

```
// IRuleEngine
function transferred(address spender, address from, address to, uint256 value)
external;

// IERC-1404
function detectTransferRestriction(address from,address to,uint256 value)
external view returns (uint8);

function messageForTransferRestriction(uint8 restrictionCode)
external view returns (string memory);

// IERC-1404Extend
enum REJECTED_CODE_BASE {
        TRANSFER_OK,
        TRANSFER_REJECTED_DEACTIVATED,
        TRANSFER_REJECTED_PAUSED,
        TRANSFER_REJECTED_FROM_FROZEN,
        TRANSFER_REJECTED_TO_FROZEN,
        TRANSFER_REJECTED_SPENDER_FROZEN,
        TRANSFER_REJECTED_FROM_INSUFFICIENT_ACTIVE_BALANCE
    }

function detectTransferRestrictionFrom(address spender,address from,address
to,uint256 value)
external view returns (uint8);


// IERC7551Compliance
function canTransferFrom(address spender,address from,address to,uint256 value)
external view returns (bool);


// IER3643ComplianceRead
function canTransfer(address from,address to,uint256 value)
external view returns (bool isValid);

// IERC3643IComplianceContract
function transferred(address from, address to, uint256 value)
external;
```

> Note: `IERC7551Compliance` comes from `draft-IERC7551` (not final) and, in this project, is used as a subset compliance interface focused on `canTransferFrom`.

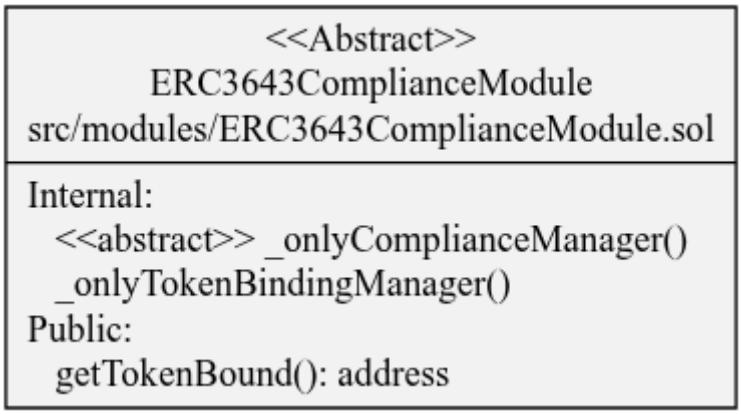
ERC-3643

The ERC-3643 compliance interface is defined in IERC3643Compliance.sol.
Non-standard helper functions are defined in IERC3643ComplianceExtended.sol.

Token binding itself is not specific to ERC-3643, so it is declared in the standard-agnostic ITokenBinding.sol (`bindToken`, `unbindToken`, `isTokenBound`, `TokenBound` / `TokenUnbound`) and ITokenBindingExtended.sol (batch binding, token self-binding, `getTokenBounds`), which the two ERC-3643 interfaces above extend.

The RuleEngine modules are split as follows:

- Token binding registry, reusable outside any compliance context: TokenBindingModule.sol and TokenBindingExtendedModule.sol
- Base ERC-3643 surface: ERC3643ComplianceModule.sol
- Non-standard extensions: ERC3643ComplianceExtendedModule.sol



Technical

Dependencies

The toolchain includes the following components, where the versions are the latest ones that we tested:

- Foundry (forge-std) v1.14.0
- Solidity 0.8.36
- OpenZeppelin Contracts (submodule) v5.6.1
- CMTAT v3.3.0

Access Control

Two access control mechanisms are available depending on which contract you deploy:

RuleEngine (RBAC - AccessControlEnumerable)

The `RuleEngine` contract uses Role-Based Access Control (RBAC) via OpenZeppelin's `AccessControlEnumerable`.

Each module defines the roles useful to restrict its functions. The contract overrides the OpenZeppelin function `hasRole` to give by default all the roles to the `admin`. `RulesManagementModule` itself is access-control agnostic; RBAC is wired at the concrete `RuleEngine` level.
Note: this `hasRole` override does not add the admin address to each role's enumerable member set. As a result, `getRoleMember` / `getRoleMemberCount` for a specific role do not include the admin unless that role is explicitly granted.

See also docs.openzeppelin.com - AccessControlEnumerable

RuleEngineOwnable (ERC-173 Ownership)

The `RuleEngineOwnable` contract uses ERC-173 ownership via OpenZeppelin's `Ownable`.

All protected functions require the caller to be the contract owner. The owner can:

- Transfer ownership to another address via `transferOwnership(address)`
- Renounce ownership via `renounceOwnership()` (makes the contract ownerless)

This is a simpler access control model suitable for single-owner deployments.

See also docs.openzeppelin.com - Ownable

RuleEngineOwnable2Step (ERC-173 Ownership, two-step transfer)

The `RuleEngineOwnable2Step` contract uses OpenZeppelin's `Ownable2Step`, which keeps the same owner-only protections and adds safer ownership handover with `transferOwnership(address)` + `acceptOwnership()`.

See also docs.openzeppelin.com - Ownable2Step

ERC-165 Support by Deployment Version

The table below summarizes which ERC-165 interfaces are advertised by each deployment version via `supportsInterface(bytes4)`.

Interface	Interface ID	RuleEngine (RBAC deployment)	RuleEngineOwnable deployment	RuleEngineOwnable2Step deployment
`IERC165`	`0x01ffc9a7`	✔	✔	✔
`IRuleEngine`	`0x20c49ce7`	✔	✔	✔
`IERC1404`	`0xab84a5c8`	✔	✔	✔
`IERC1404Extend`	`0x78a8de7d`	✔	✔	✔
`IERC3643Compliance`	`0x3144991c`	✔	✔	✔
`IERC3643ComplianceExtended`	`0x646ba2be`	✔	✔	✔
`IERC7551Compliance` (subset)	`0x7157797f`	✔	✔	✔
`IERC173`	`0x7f5828d0`	✘	✔	✔
`Ownable2Step` specific (`pendingOwner()`, `acceptOwnership()`)	`0x9ab669ef`	✘	✘	✔

Interface	Interface ID	RuleEngine (RBAC deployment)	RuleEngineOwnable deployment	RuleEngineOwnable2Step deployment
`IAccessControl`	`0x7965db0b`	✔	✘	✘
`IAccessControlEnumerable`	`0x5a05180f`	✔	✘	✘

The six interfaces common to all three variants come from `RuleEngineBase._supportsRuleEngineBaseInterface`;
each deployable adds the IDs of its own access-control model on top.

Notes:

- `RuleEngine` advertises OpenZeppelin RBAC interfaces because it inherits `AccessControlEnumerable`.
- `RuleEngineOwnable` / `RuleEngineOwnable2Step` intentionally do not advertise `IAccessControl`.
- `Ownable2Step` specific interface ID is defined in `Ownable2StepInterfaceId` and includes only `pendingOwner()` and `acceptOwnership()`.
- **`IRule` (`0x2497d6cb`) is deliberately absent from this table.** The engine does not implement `IRule`; it
  requires it of every rule, checking it in `_checkRule` before a rule is added. Do not expect `ruleEngine.supportsInterface(0x2497d6cb)` to return true.
- **The IDs are computed from the interfaces, not hardcoded** — see `ComplianceInterfaceId`, `RuleInterfaceId`
  and `ERC1404InterfaceId`, and `test/RuleEngine/IRuleInterfaceId.t.sol`, which pins each one to the wire value
  above. `type(I).interfaceId` counts only the functions `I` declares *directly*, so every constant XORs the
  interface with its parents.
- **Never use `type(IERC3643ComplianceExtended).interfaceId`.** That interface declares no function of its own —
  it only combines `IERC3643Compliance` and `ITokenBindingExtended` — so the expression evaluates to
  `0x00000000`. Use
  `ComplianceInterfaceId.ERC3643_COMPLIANCE_EXTENDED_INTERFACE_ID` (`0x646ba2be`), which is
  computed from `ITokenBindingExtended`.

Role list (RuleEngine only)

Here is the list of roles and their 32 bytes identifier for the `RuleEngine` contract.

The default admin is the address put in argument (`admin`) inside the constructor.

It is set in the constructor when the contract is deployed.

> Note: For `RuleEngineOwnable` and `RuleEngineOwnable2Step`, all protected functions are controlled by the single `owner` address instead of roles.

> **Warning (role assignment):** Rule contracts should be treated as trusted logic components and kept separate from governance/operator identities. The protocol now enforces key protections on-chain: in RBAC deployments, `grantRole` reverts if the target account is in the rule set; in ownable deployments, `transferOwnership` reverts if the new owner is in

> the rule set. In multi-token deployments, do not grant any governance/operator privileges to token contract addresses (bound tokens should remain data-plane callers only, meaning runtime compliance callbacks such as `transferred`, `created`, and `destroyed`). This token-privilege separation is intentionally documented as an operational constraint (not enforced on-chain) to preserve flexibility for integrators who explicitly want to extend their token and route selected RuleEngine control-plane actions through token logic (`control-plane` here means configuration/governance actions such as `bindToken`, `unbindToken`, role grants, ownership changes, and rule management).

	Defined in	32 bytes identifier
DEFAULT_ADMIN_ROLE	OpenZeppelin AccessControl	0x0000000000000000000000000000000000000000000000000000000000000000
Modules		
COMPLIANCE_MANAGER_ROLE	ERC3643ComplianceRolesStorage	0xe5c50d0927e06141e032cb9a67e1d7092dc85c0b0825191f7e1cede600028568
RULES_MANAGEMENT_ROLE	RulesManagementModuleInvariantStorage	0xea5f4eb72290e50c32abd6c23e45de3d8300b3286e1cbc2e293114b92e034e5e

Schema (RuleEngine)

Here is a schema of the Access Control for `RuleEngine`.

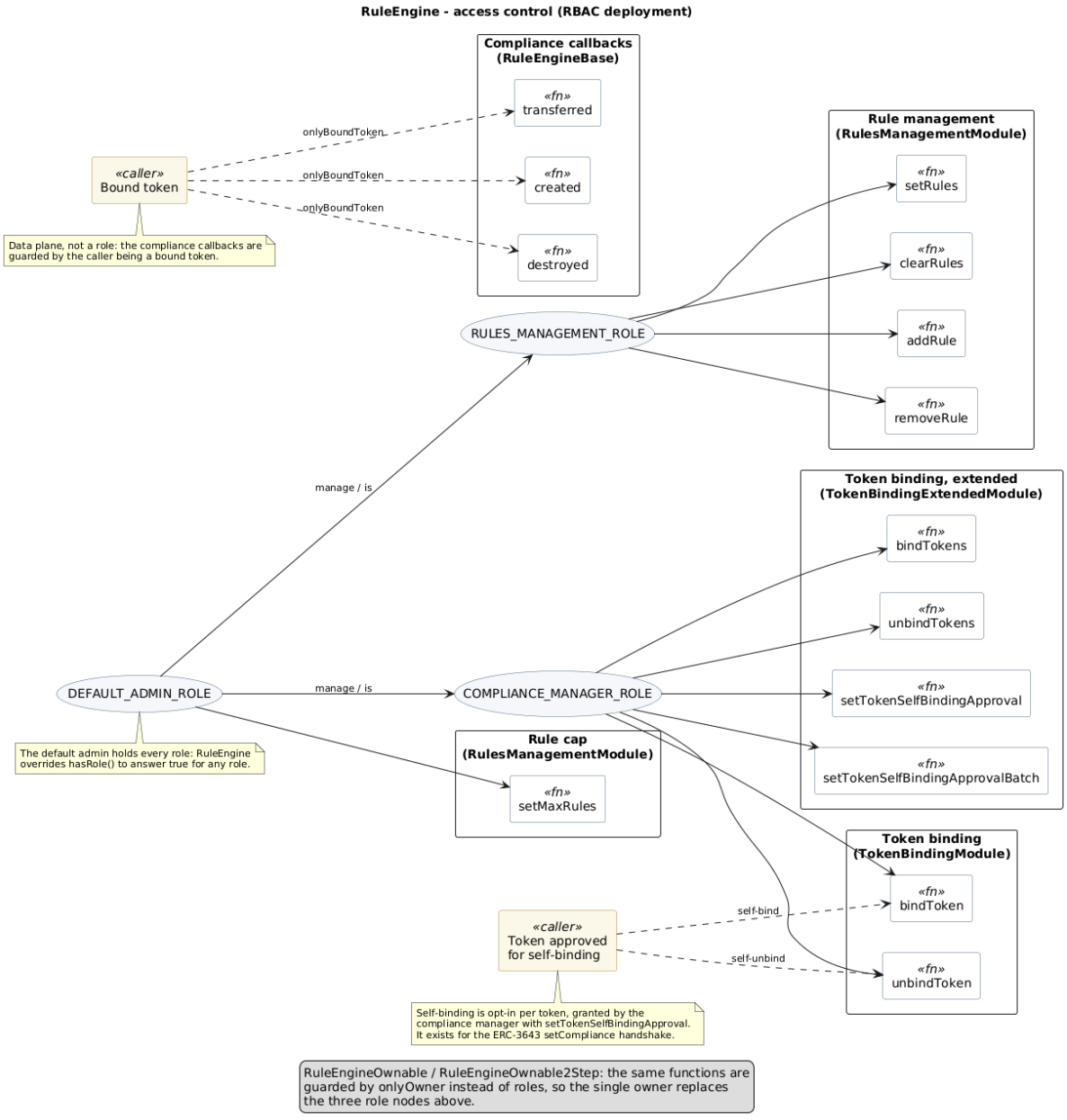


Diagram source: doc/schema/plantuml/ruleengine-access-control.puml.

Role by modules (RuleEngine)

Here is a summary table for each restricted function defined in a module.
For function signatures, struct arguments are represented with their corresponding native type.

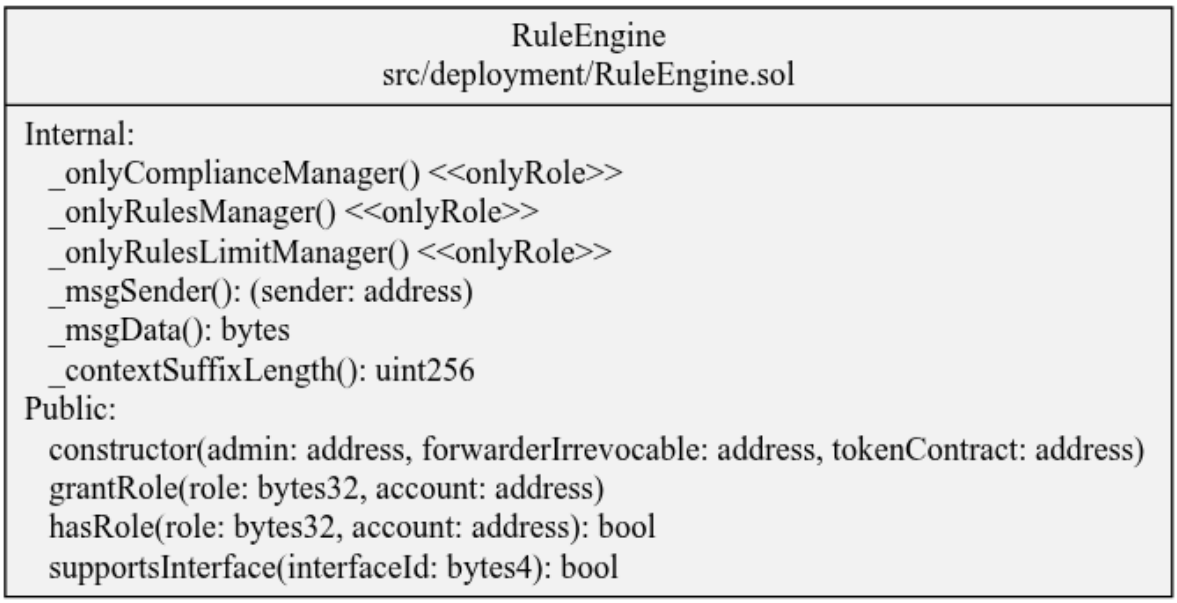
> Note: For `RuleEngineOwnable` and `RuleEngineOwnable2Step`, replace the role requirement with `onlyOwner` for all protected functions.

	Function signature	Visibility [public/external]	Input variables (Function arguments)	Output variables (return value)	Role Required
Modules					
RulesManagementModule					
	`setRules(address[] rules_)`	public	`IRule[] rules_`	-	RULES_MANAGEMENT_ROLE
	`clearRules()`	public	-	-	RULES_MANAGEMENT_ROLE
	`addRule(address rule_)`	public	`IRule rule_`	-	RULES_MANAGEMENT_ROLE
	`removeRule(address rule_)`	public	`IRule rule_`	-	RULES_MANAGEMENT_ROLE
	`setMaxRules(uint256 maxRules_)`	public	`uint256 maxRules_`	-	DEFAULT_ADMIN_ROLE
TokenBindingModule					
	`bindToken(address token)`	public	`address token`	-	COMPLIANCE_MANAGER_ROLE or approved token self-call
	`unbindToken(address token)`	public	`address token`	-	COMPLIANCE_MANAGER_ROLE or approved token self-call
TokenBindingExtendedModule					
	`bindTokens(address[] tokens)`	public	`address[] tokens`	-	COMPLIANCE_MANAGER_ROLE
	`unbindTokens(address[] tokens)`	public	`address[] tokens`	-	COMPLIANCE_MANAGER_ROLE
	`setTokenSelfBindingApproval(address token,bool approved)`	public	`address token,bool approved`	-	COMPLIANCE_MANAGER_ROLE
	`setTokenSelfBindingApprovalBatch(address[] tokens,bool approved)`	public	`address[] tokens,bool approved`	-	COMPLIANCE_MANAGER_ROLE
RuleEngineBase					
	`transferred(address from,address to,uint256 value)`	public	`address from,address to, uint256 value`	-	onlyBoundToken (modifier)
	`transferred(address spender,address from,address to,uint256 value)`	public	`address spender,address from,address to, uint256 value`	-	onlyBoundToken (modifier)

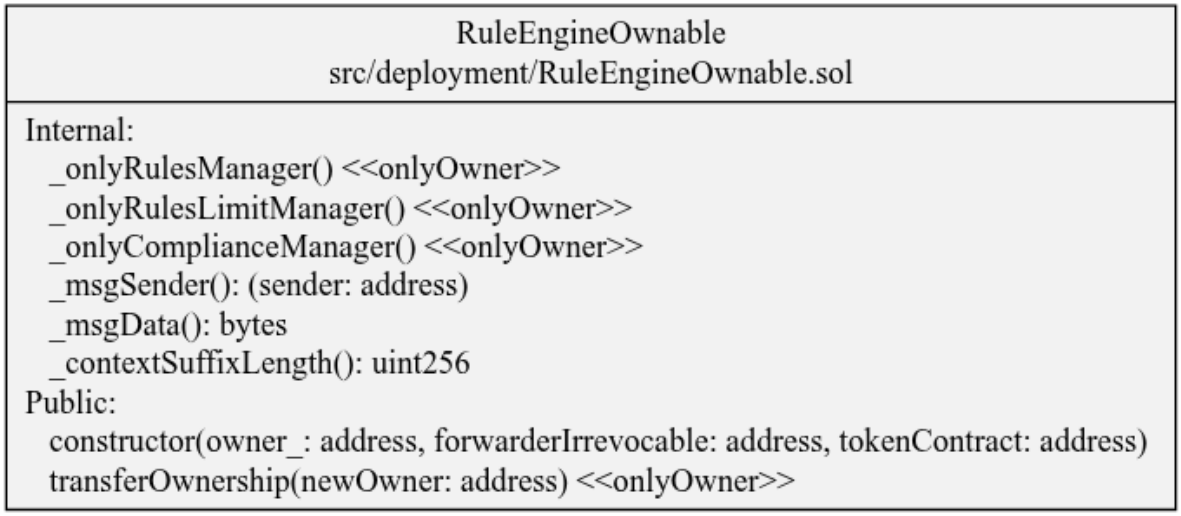
UML

Here is the UML of the main contracts. The diagrams are generated with sol2uml, one per contract or interface, by doc/script/script_sol2uml.sh; rerun that script after changing a contract's surface.

RuleEngine



RuleEngineOwnable



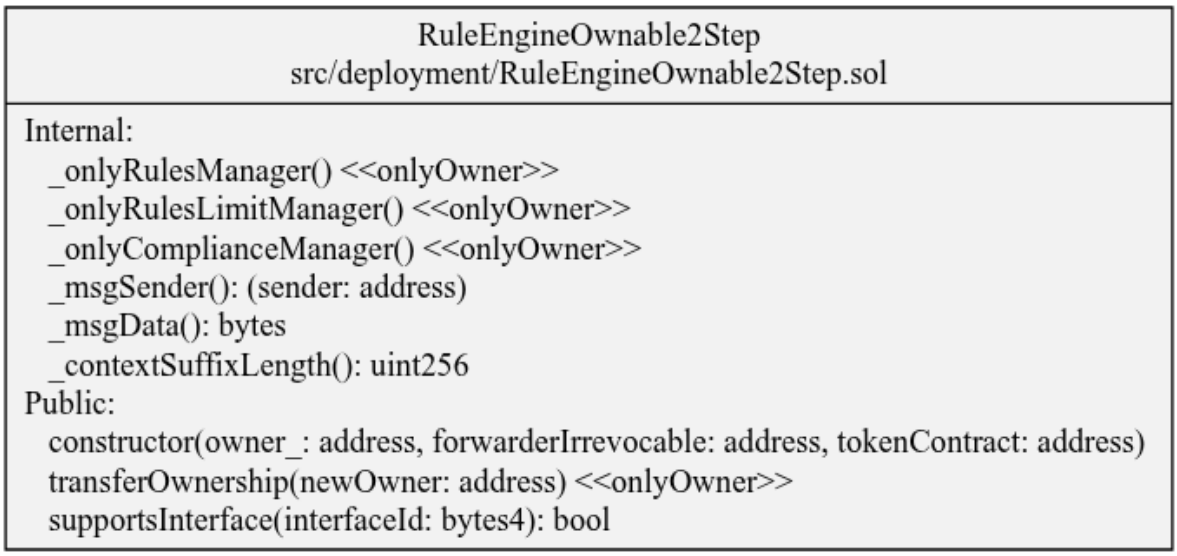
`RuleEngineOwnable` shares the same base functionality as `RuleEngine` but uses ERC-173 ownership instead of RBAC.

```
RuleEngineOwnable
├── ERC2771ModuleStandalone (gasless support)
├── RuleEngineBase (core functionality)
│   ├── VersionModule
│   ├── RulesManagementModule
│   ├── ERC3643ComplianceModule (core ERC-3643)
│   ├── ERC3643ComplianceExtendedModule (project extensions)
│   └── IRuleEngineERC1404
└── Ownable (ERC-173 access control)
```

Key differences from RuleEngine:

- Constructor takes `owner_` instead of `admin`
- All protected functions use `onlyOwner` modifier
- Supports `transferOwnership()` and `renounceOwnership()`
- Implements ERC-173 interface (`supportsInterface(0x7f5828d0)` returns `true`)

RuleEngineOwnable2Step



`RuleEngineOwnable2Step` shares the same base functionality as `RuleEngineOwnable` but uses OpenZeppelin's `Ownable2Step` for safer ownership handover.

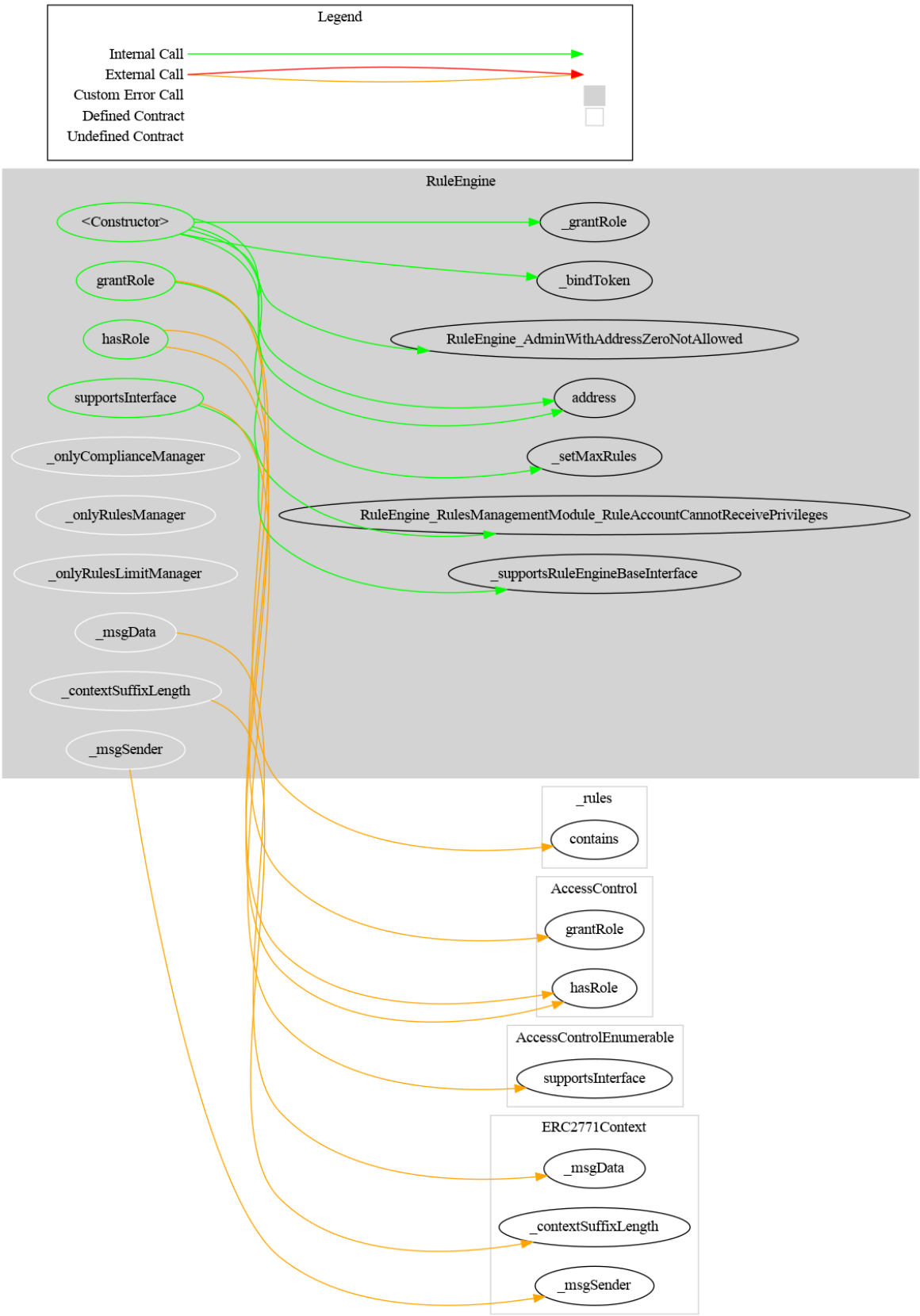
```
RuleEngineOwnable2Step
├── ERC2771ModuleStandalone (gasless support)
├── RuleEngineOwnableShared (shared ownable deployment logic)
│   └── RuleEngineBase
│       ├── VersionModule
│       ├── RulesManagementModule
│       ├── ERC3643ComplianceModule (core ERC-3643)
│       ├── ERC3643ComplianceExtendedModule (project extensions)
│       └── IRuleEngineERC1404
└── Ownable2Step (ERC-173 access control with pending owner)
```

Key differences from RuleEngineOwnable:

- Uses a two-step ownership transfer flow: `transferOwnership()` then `acceptOwnership()`
- The current owner retains privileges until the pending owner accepts ownership
- Reuses `RuleEngineOwnableShared` for constructor, ERC-165 (via OpenZeppelin `ERC165`), and ERC-2771 behavior
- Implements ERC-173 interface (`supportsInterface(0x7f5828d0)` returns `true`)
- Implements Ownable2Step-specific ERC-165 interface (`supportsInterface(0x9ab669ef)` returns `true`), covering `pendingOwner()` and `acceptOwnership()`

Graph

Here is the surya graph of the main contract:



Functionality

Several functionalities are not implemented because it makes more sense to directly implement them in the token smart contract

The RuleEngine can be removed from the main token contract by calling these dedicated functions

- CMTAT v3.0.0: `setRuleEngine(address ruleEngine)`
- ERC-3643 token: `setCompliance(address _compliance)`

Available Rules

Rules are maintained in a dedicated repository: github.com/CMTA/Rules.

Rules can be used in two ways:

- Directly on CMTAT (single-rule setup, no RuleEngine orchestration).
- Through this RuleEngine (multi-rule orchestration with sequential execution).

Rule families:

Family	Behavior	Examples
Validation rules (read-only)	Evaluate transfer eligibility without mutating rule state	`RuleWhitelist`, `RuleBlacklist`, `RuleSanctionList`, `RuleIdentityRegistry`, `RuleSpenderWhitelist`, `RuleERC2980`, `RuleMaxTotalSupply`
Operation rules (read-write)	Evaluate transfer eligibility and can update rule-specific state on transfer	`RuleConditionalTransferLight`

Additional integration notes:

- For RuleEngine integration, a rule must implement `IRule` (including ERC-165 support for the Rule interface ID).
- RuleEngine executes configured rules in order and reverts on the first failing rule in state-changing paths.
- Restriction codes should remain unique across the composed rule set. Keep CMTAT-reserved ranges free and use dedicated code ranges per rule
- For the latest list of production rules, audits, and status, use the Rules repository as the source of truth.

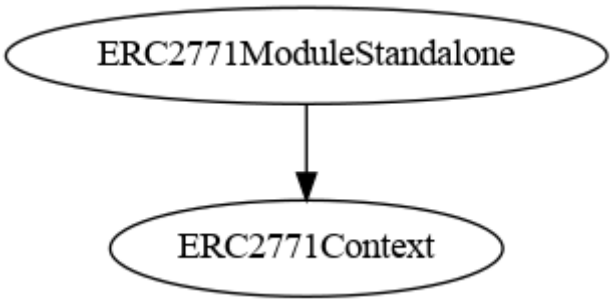
Rules details

Here is a summary tab of available rules, see github.com/CMTA/Rules

Rule	Type [read-only / read-write]	Description
RuleWhitelist	Read-only	This rule can be used to restrict transfers from/to only addresses inside a whitelist.
RuleWhitelistWrapper	Read-Only	This rule can be used to restrict transfers from/to only addresses inside a group of whitelist rules managed by different operators.
RuleBlacklist	Read-Only	This rule can be used to forbid transfer from/to addresses in the blacklist

Rule	Type [read-only / read-write]	Description
RuleSanctionList	Read-Only	The purpose of this contract is to use the oracle contract from Chainalysis to forbid transfer from/to an address included in a sanctions designation (US, EU, or UN).
RuleMaxTotalSupply	Read-Only	This rule limits minting so that the total supply never exceeds a configured maximum.
RuleIdentityRegistry	Read-Only	This rule checks the ERC-3643 Identity Registry for transfer participants when configured.
RuleSpenderWhitelist	Read-Only	This rule blocks `transferFrom` when the spender is not in the whitelist. Direct transfers are always allowed.
RuleERC2980	Read-Only	ERC-2980 Swiss Compliant rule combining a whitelist (recipient-only) and a frozenlist (blocks sender, recipient, and spender for `transferFrom`). Frozenlist takes priority over whitelist.
RuleConditionalTransferLight	Read-Write	This rule requires that transfers have to be approved by an operator before being executed. Each approval is consumed once and the same transfer can be approved multiple times.
RuleConditionalTransfer (external)	Read-Write	Full-featured approval-based transfer rule implementing Swiss law *Vinkulierung*. Supports automatic approval after three months, automatic transfer execution, and a conditional whitelist for address pairs that bypass approval. Maintained in a separate repository.
RuleSelf (community)	—	Use Self, a zero-knowledge identity solution to determine which is allowed to interact with the token. Community-maintained rule project. Not developed or maintained by CMTA.

Gasless support (ERC-2771)



The RuleEngine supports client-side gasless transactions using the standard ERC-2771.

The contract uses the OpenZeppelin contract `ERC2771ContextUpgradeable`, which allows a contract to get the original client with `_msgSender()` instead of the feepayer given by `msg.sender`.

At deployment, the parameter `forwarder` inside the RuleEngine contract constructor has to be set with the defined address of the forwarder.

After deployment, the forwarder is immutable and can not be changed.

References:

- OpenZeppelin Meta Transactions
- OpenGSN has deployed several forwarders, see their documentation for examples.

Upgradeable

A proxy architecture (upgradeable) increases the code complexity as well as the runtime gas cost for each transaction. This is why the RuleEngine is not upgradeable.

Moreover, in a proxy architecture, each new implementation must be compatible (storage) with the precedent implementation, which can reduce the ability to improve the code.

In case you use the same RuleEngine for several different tokens, unfortunately, you will have to update the address of the RuleEngine set in each token contract separately.

Urgency mechanism

Pause

There are no functionalities to put the RuleEngine in pause .

The RuleEngine can be removed from the main token contract by calling the dedicated functions to manage the RuleEngine

Kill / Deactivate the contracts

There are no functionalities to kill/deactivate the contracts.

Similar to the pause functionality, the RuleEngine can be directly removed from the main token contract.

Ethereum API

Contract Constructors

RuleEngine Constructor

```
constructor(
    address admin,
    address forwarderIrrevocable,
    address tokenContract
)
```

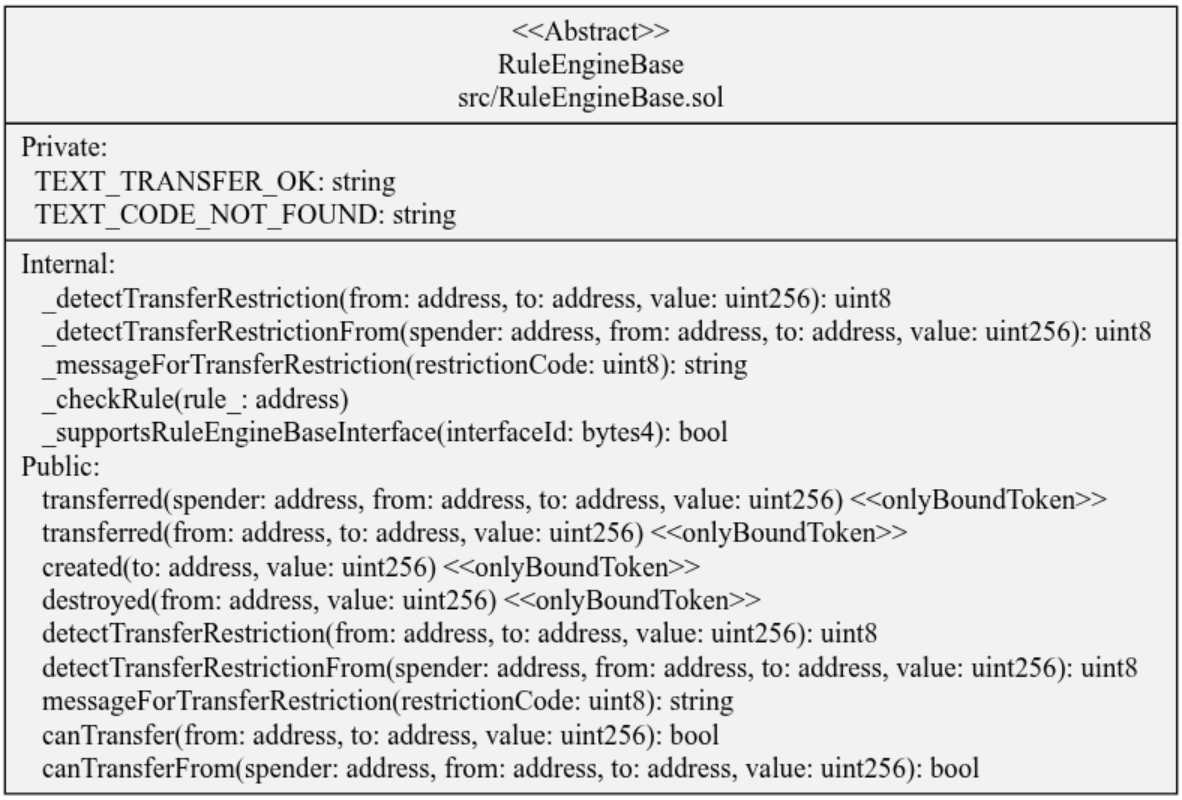
Parameter	Type	Description
admin	address	Address granted DEFAULT_ADMIN_ROLE (has all roles)
forwarderIrrevocable	address	ERC-2771 trusted forwarder address (can be zero)
tokenContract	address	Token to bind at deployment (can be zero)

RuleEngineOwnable Constructor

```
constructor(
    address owner_,
    address forwarderIrrevocable,
    address tokenContract
)
```

Parameter	Type	Description
owner_	address	Address set as contract owner (ERC-173)
forwarderIrrevocable	address	ERC-2771 trusted forwarder address (can be zero)
tokenContract	address	Token to bind at deployment (can be zero)

RuleEngineBase



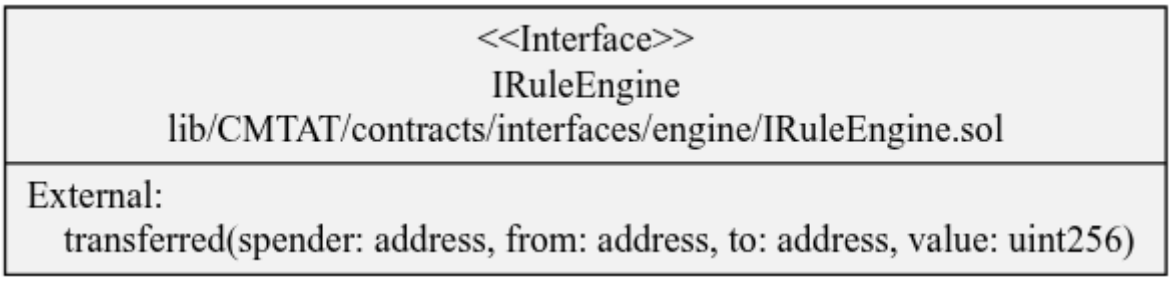
Contracts Description Table

Contract	Type	Bases		
└	**Function Name**	**Visibility**	**Mutability**	**Modifiers**
RuleEngineBase	Implementation	VersionModule, RulesManagementModule, ERC3643ComplianceExtendedModule, RuleEngineInvariantStorage, IRuleEngine		
└	transferred	Public ❗	🛑	onlyBoundToken
└	transferred	Public ❗	🛑	onlyBoundToken
└	created	Public ❗	🛑	onlyBoundToken
└	destroyed	Public ❗	🛑	onlyBoundToken
└	detectTransferRestriction	Public ❗		NO ❗
└	detectTransferRestrictionFrom	Public ❗		NO ❗
└	canTransfer	Public ❗		NO ❗
└	canTransferFrom	Public ❗		NO ❗
└	messageForTransferRestriction	Public ❗		NO ❗
└	hasRole	Public ❗		NO ❗

Legend

Symbol	Meaning
🛑	Function can modify state
💵	Function is payable

IRuleEngine



transferred(address spender, address from, address to, uint256 value)

```
function transferred(address spender,address from,address to,uint256 value)
public virtual override(IRuleEngine)
onlyBoundToken
```

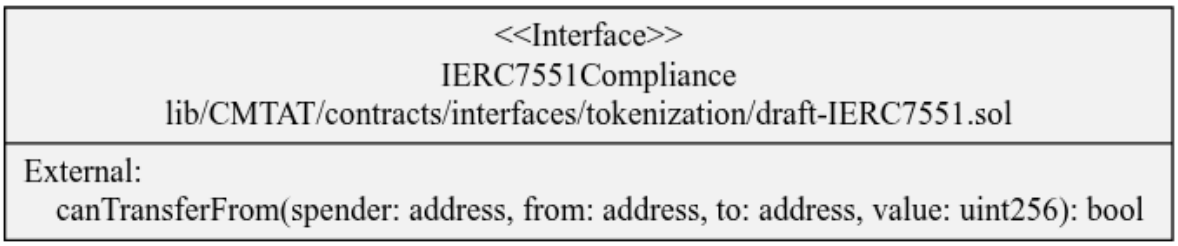
Function called whenever tokens are transferred from one wallet to another.

Must revert if the transfer is invalid.
Same name as ERC-3643 but with an additional `spender` parameter.
This function can be used to update state variables of the RuleEngine contract.
Can only be called by the token contract bound to the RuleEngine.

Input Parameters:

Name	Type	Description
spender	address	The spender address initiating the transfer.
from	address	The token holder address.
to	address	The receiver address.
value	uint256	The amount of tokens involved in the transfer.

IERC7551Compliance



> Note: ERC-7551 is draft (not final). The `IERC7551Compliance` interface used here is a subset interface exposing the compliance check `canTransferFrom`.

canTransferFrom(address spender, address from, address to, uint256 value) -> bool

Checks if `spender` can transfer `value` tokens from `from` to `to` under compliance rules.

Does not check balances or access rights (Access Control).

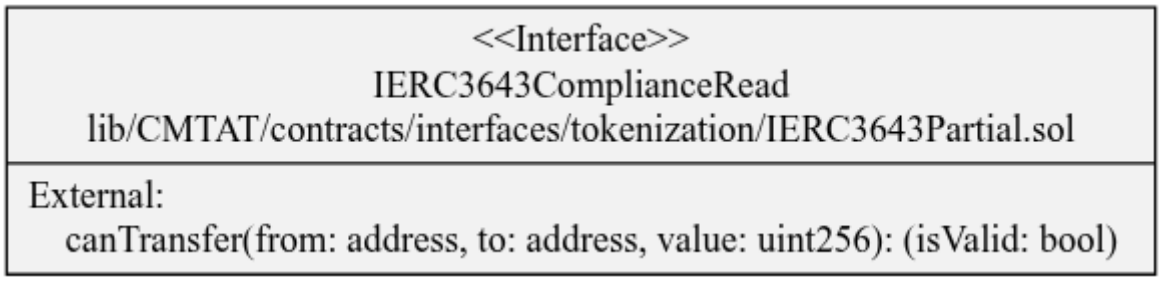
Input Parameters:

Name	Type	Description
spender	address	The address performing the transfer.
from	address	The source address.
to	address	The destination address.
value	uint256	The number of tokens to transfer.

Return Values:

Type	Description
bool	True if the transfer complies with policy.

IERC3643ComplianceRead



---

canTransfer(address from, address to, uint256 value) -> bool

Returns true if the transfer is valid, and false otherwise.

Does not check balances or access rights (Access Control).

> **Warning (spender-dependent rules fail open here):** this signature carries no `spender`. A rule whose
> decision depends on the spender — for example a per-minter mint allowance — cannot evaluate the operation
> on this path and must answer "no restriction". The RuleEngine aggregates that answer, so `canTransfer` and
> `detectTransferRestriction` can report a mint as allowed that the state-changing `transferred(spender, from, to, value)` will revert. To pre-check an operation that has an operator, use
> the 4-argument `canTransferFrom` / `detectTransferRestrictionFrom`.

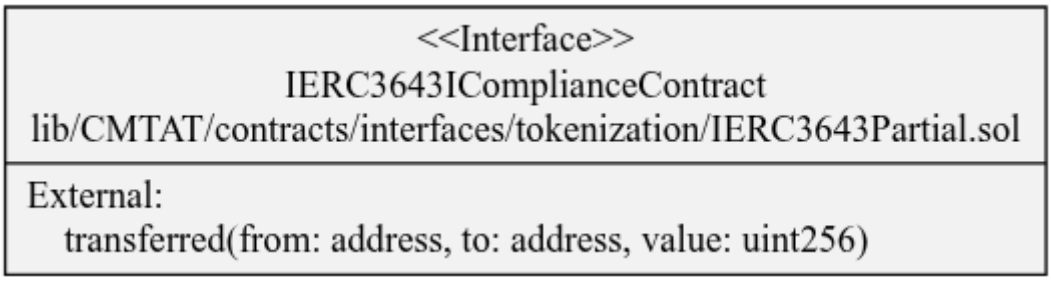
Input Parameters:

Name	Type	Description
from	address	The source address.
to	address	The destination address.
value	uint256	The number of tokens to transfer.

Return Values:

Type	Description
bool	True if the transfer is valid, false otherwise.

IERC3643IComplianceContract



---

transferred(address from, address to, uint256 value)

```
function transferred(address from,address to,uint256 value)
public virtual override(IERC3643IComplianceContract)
onlyBoundToken
```

Updates the compliance contract state whenever tokens are transferred.

Can only be called by the token contract bound to this compliance logic.
This function can be used to update internal state variables.

Input Parameters:

Name	Type	Description
from	address	The address of the sender.
to	address	The address of the receiver.
value	uint256	The number of tokens involved in the transfer.

IERC3643Compliance

---

created(address to, uint256 value)

```
function created(address to, uint256 value)
public virtual override(IERC3643Compliance)
onlyBoundToken
```

Updates the compliance contract state when tokens are created (minted).

Called by the token contract when new tokens are issued to an account.
Reverts if the minting does not comply with the rules.

Input Parameters:

Name	Type	Description
to	address	The address receiving the minted tokens.
value	uint256	The number of tokens created.

---

destroyed(address from, uint256 value)

```
function destroyed(address from, uint256 value)
public virtual override(IERC3643Compliance)
onlyBoundToken
```

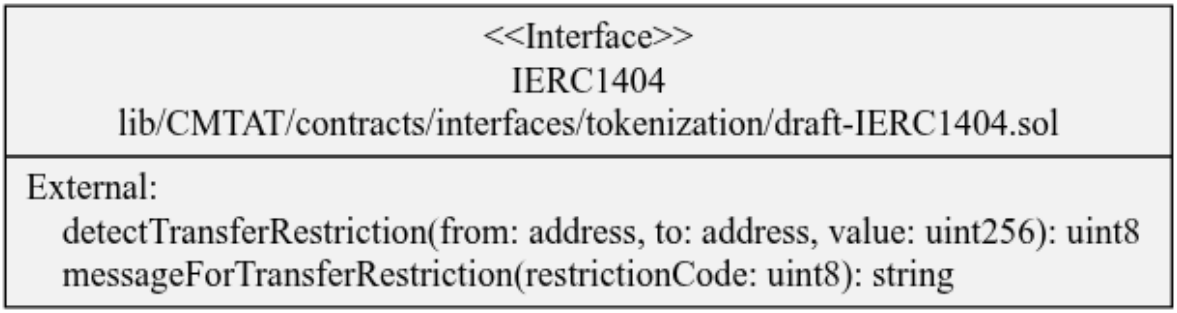
Updates the compliance contract state when tokens are destroyed (burned).

Called by the token contract when tokens are redeemed or burned.
Reverts if the burning does not comply with the rules.

Input Parameters:

Name	Type	Description
from	address	The address whose tokens are being destroyed.
value	uint256	The number of tokens destroyed.

IERC1404



---

detectTransferRestriction(address from, address to, uint256 value) -> uint8

Returns a uint8 code to indicate if a transfer is restricted or not.

Implements the restriction logic of {ERC-1404}.
Examples of restriction logic include:

- checking if the recipient is whitelisted,
- checking if the sender's tokens are frozen during a lock-up period, etc.

Input Parameters:

Name	Type	Description
from	address	The source address.
to	address	The destination address.
value	uint256	The number of tokens to transfer.

Return Values:

Type	Description
uint8	Restriction code (0 means the transfer is authorized).

---

messageForTransferRestriction(uint8 restrictionCode) -> string

Returns a human-readable explanation for a transfer restriction code.

Implements {ERC-1404} standard message accessor.

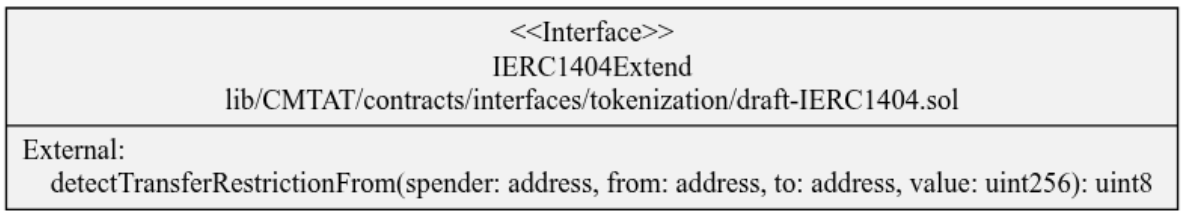
Input Parameters:

Name	Type	Description
restrictionCode	uint8	The restriction code to interpret.

Return Values:

Type	Description
string	A message describing why the transfer is restricted.

IERC1404Extend



enum REJECTED_CODE_BASE

Error codes for transfer restrictions.
Codes `6-9` are reserved for future CMTAT ruleEngine extensions.

Name	Value	Description
TRANSFER_OK	0	Transfer authorized.
TRANSFER_REJECTED_PAUSED	1	Transfer rejected because the token is paused.
TRANSFER_REJECTED_FROM_FROZEN	2	Transfer rejected because the sender's address is frozen.
TRANSFER_REJECTED_TO_FROZEN	3	Transfer rejected because the recipient's address is frozen.
TRANSFER_REJECTED_SPENDER_FROZEN	4	Transfer rejected because the spender's address is frozen.
TRANSFER_REJECTED_FROM_INSUFFICIENT_ACTIVE_BALANCE	5	Transfer rejected because the sender does not have enough active (unfrozen) balance.

---

detectTransferRestrictionFrom(address spender, address from, address to, uint256 value) -> uint8

Returns a uint8 code to indicate if a transfer is restricted or not.

This is an extension of {ERC-1404} with an additional `spender` parameter to enforce restriction logic on delegated transfers.
Examples of restriction logic include:

- verifying if the recipient is whitelisted,
- verifying if tokens are locked for either sender or spender, etc.

Input Parameters:

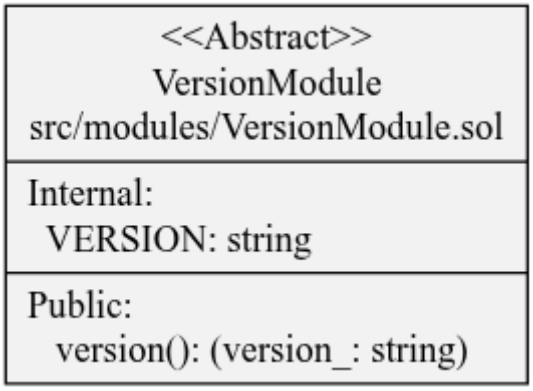
Name	Type	Description
spender	address	The address initiating the transfer (for delegated transfers).

Name	Type	Description
from	address	The source address.
to	address	The destination address.
value	uint256	The number of tokens to transfer.

Return Values:

Type	Description
uint8	Restriction code (0 means the transfer is authorized).

VersionModule



Contracts Description Table

Contract	Type	Bases		
L	**Function Name**	**Visibility**	**Mutability**	**Modifiers**
VersionModule	Implementation	IERC3643Base		
L	version	Public ❗		NO ❗

version()

```
function version() external view returns (string memory version_);
```

```
function version()
public view virtual override(IERC3643Base)
returns (string memory version_)
```

Description

Returns the current version of the token contract.
Useful for identifying which version of the smart contract is deployed and in use.

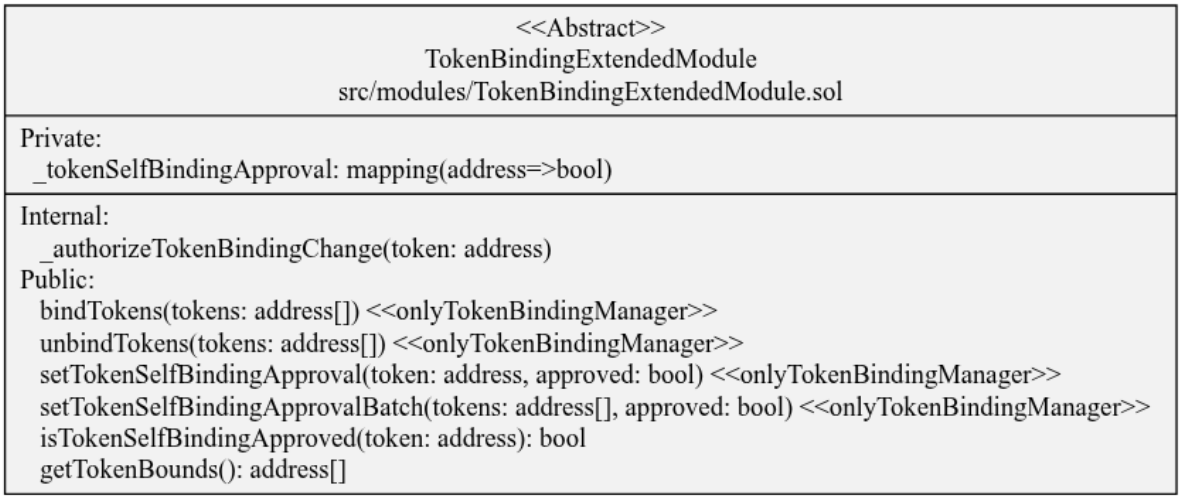
Return

Name	Type	Description
`version_`	string	The version string of the token implementation (e.g., "1.0.0").

TokenBindingModule



`TokenBindingModule` holds the token binding registry itself: the set of tokens allowed to call the bound-token entry points, `bindToken` / `unbindToken` / `isTokenBound`, and the `onlyBoundToken` guard. It is standard-agnostic — it contains no ERC-3643, ERC-1404 or rule logic and depends only on OpenZeppelin's `Context` and `EnumerableSet` — so it can be reused as-is by any project that has to bind tokens. A deployment only has to provide the access control by implementing `_onlyTokenBindingManager()`; `src/mocks/TokenBindingStandaloneMock.sol` is a minimal example of such a reuse, outside any compliance context.



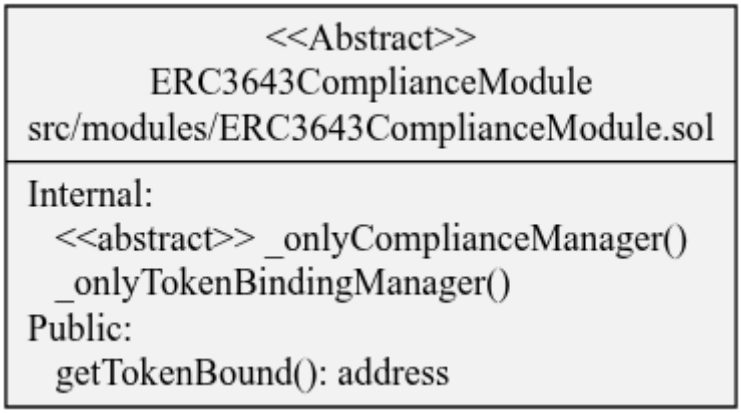
`TokenBindingExtendedModule` adds the conveniences of the registry that are not part of any token
standard: batch bind/unbind, token self-binding approval (used by ERC-3643 `setCompliance`), and
`getTokenBounds()`. It also replaces the default binding authorization with one that accepts an approved token binding itself.

The events and functions of both modules are documented in the events and functions reference below,
under `ERC3643ComplianceExtendedModule`, since that is the form in which the RuleEngine exposes them.

Contracts Description Table

Contract	Type	Bases		
└	**Function Name**	**Visibility**	**Mutability**	**Modifiers**
TokenBindingModule	Implementation	Context, ITokenBinding		
└	bindToken	Public ❗	🛑	NO ❗
└	unbindToken	Public ❗	🛑	NO ❗
└	isTokenBound	Public ❗		NO ❗
└	_bindToken	Internal 🔒	🛑	
└	_unbindToken	Internal 🔒	🛑	
TokenBindingExtendedModule	Implementation	TokenBindingModule, ITokenBindingExtended		
└	bindTokens	Public ❗	🛑	onlyTokenBindingManager
└	unbindTokens	Public ❗	🛑	onlyTokenBindingManager
└	setTokenSelfBindingApproval	Public ❗	🛑	onlyTokenBindingManager
└	setTokenSelfBindingApprovalBatch	Public ❗	🛑	onlyTokenBindingManager
└	isTokenSelfBindingApproved	Public ❗		NO ❗
└	getTokenBounds	Public ❗		NO ❗

ERC3643ComplianceModule

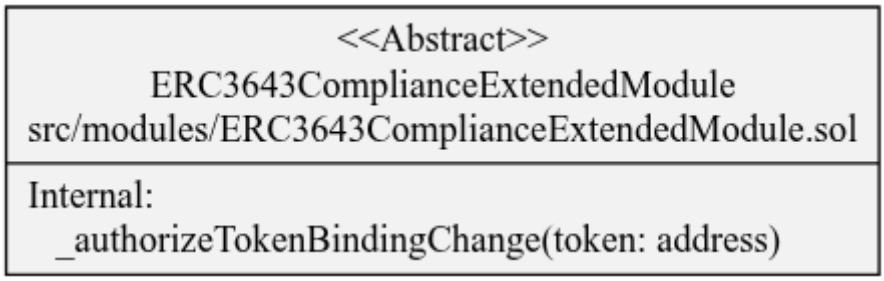


`ERC3643ComplianceModule` is a thin ERC-3643 adapter over `TokenBindingModule`: it adds the ERC-3643 specific view `getTokenBound()` and names the binding manager in compliance terms, wiring
the generic `_onlyTokenBindingManager()` hook to `_onlyComplianceManager()`, which the deployable
contracts implement (`COMPLIANCE_MANAGER_ROLE` for `RuleEngine`, `onlyOwner` for the ownable variants). The compliance callbacks `transferred`, `created` and `destroyed` are implemented by `RuleEngineBase`, since they depend on the rules rather than on the binding.

Contracts Description Table

Contract	Type	Bases		
└	**Function Name**	**Visibility**	**Mutability**	**Modifiers**
ERC3643ComplianceModule	Implementation	TokenBindingModule, IERC3643Compliance		
└	getTokenBound	Public ❗		NO ❗
└	_onlyTokenBindingManager	Internal 🔒	🛑	

ERC3643ComplianceExtendedModule



`ERC3643ComplianceExtendedModule` combines `ERC3643ComplianceModule` with `TokenBindingExtendedModule` and declares `IERC3643ComplianceExtended`. It carries no logic of its own: the project-specific helpers that are not part of the ERC-3643 base interface (`IERC3643Compliance`) — batch bind/unbind, self-binding approval APIs and `getTokenBounds()` — are standard-agnostic and therefore implemented in `TokenBindingExtendedModule`.

Events

TokenBound(address token)

```
event TokenBound(address token)
```

Emitted when a token is successfully bound to the compliance contract.

Event Parameters:

Name	Type	Description
token	address	The address of the token that was bound.

TokenUnbound(address token)

```
event TokenUnbound(address token)
```

Emitted when a token is successfully unbound from the compliance contract.

Event Parameters:

Name	Type	Description
token	address	The address of the token that was unbound.

Functions

bindToken(address token)

```
function bindToken(address token)
public override virtual
```

Associates a token contract with this compliance contract.

Implemented by `TokenBindingModule`. Authorization goes through `_authorizeTokenBindingChange`, which
accepts the compliance manager (`COMPLIANCE_MANAGER_ROLE` for `RuleEngine`, the owner for the ownable
variants), or the token itself when its self-binding has been approved.

The compliance contract may restrict operations on the bound token according to its internal compliance logic.
Reverts if the token is already bound.

Input Parameters:

Name	Type	Description

Name	Type	Description
token	address	The address of the token to bind.

---

unbindToken(address token)

```
function unbindToken(address token)
public override virtual
```

Removes the association of a token contract from this compliance contract.

Implemented by `TokenBindingModule`, with the same authorization as `bindToken`.

Reverts if the token is not currently bound.

Input Parameters:

Name	Type	Description
token	address	The address of the token to unbind.

---

isTokenBound(address token) -> bool

```
function isTokenBound(address token)
public view virtual override
returns (bool)
```

Checks whether a token is currently bound to this compliance contract.

Input Parameters:

Name	Type	Description
token	address	The token address to verify.

Return Values:

Type	Description
bool	True if the token is bound, false otherwise.

---

getTokenBound() -> address

```
function getTokenBound()
public view virtual override
returns (address)
```

Returns the single token currently bound to this compliance contract.

If multiple tokens are supported, consider using `getTokenBounds()`.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Return Values:

Type	Description
address	The address of the currently bound token.

---

getTokenBounds() -> address[]

```
function getTokenBounds()
public view override
returns (address[] memory)
```

Returns all tokens currently bound to this compliance contract.

This is a view-only function and does not modify state.
This function is not part of the original ERC-3643 specification.

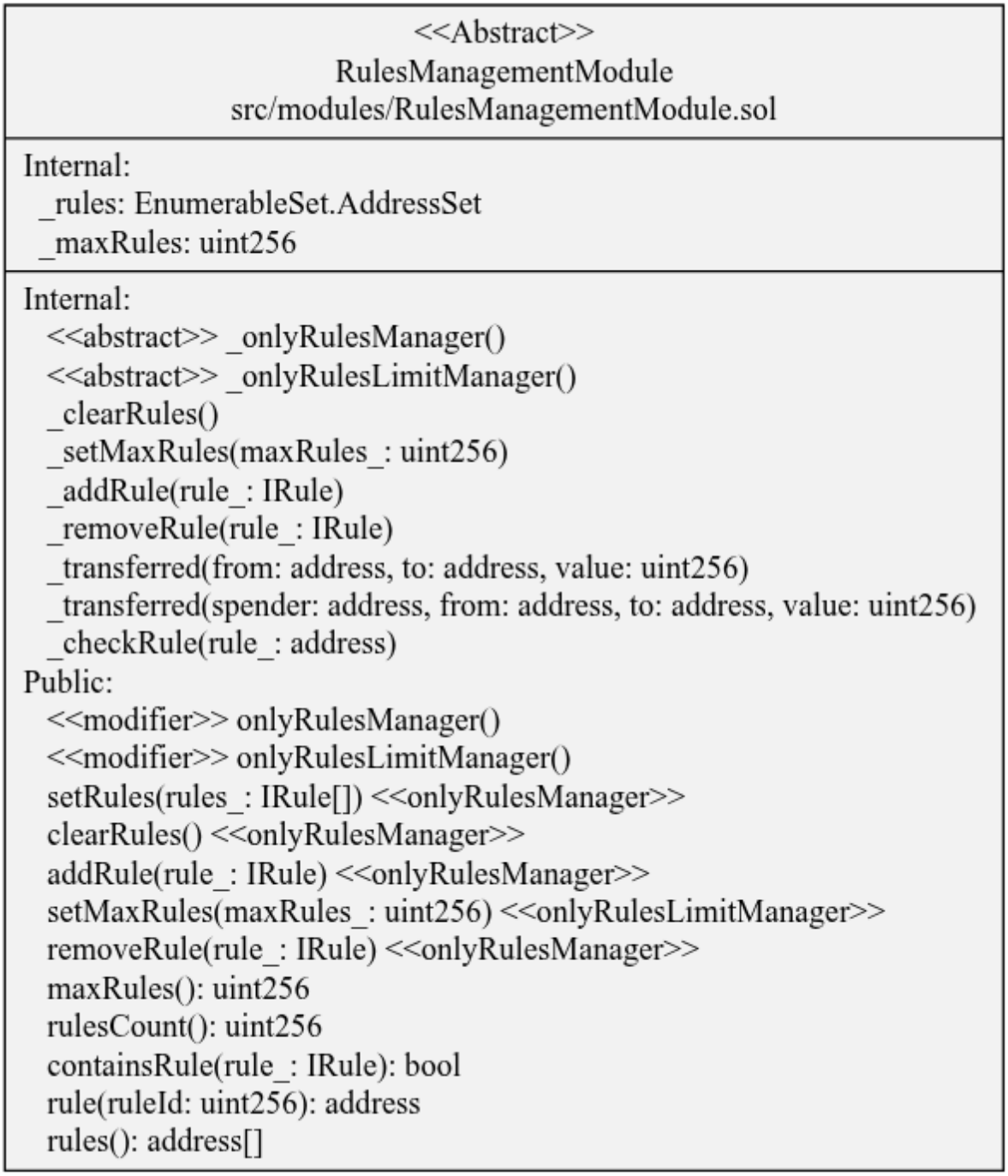
This operation will copy the entire storage to memory, which can be quite expensive.

This is designed to mostly be used by view accessors that are queried without any gas fees.

Return Values:

Type	Description
address[]	An array of addresses of bound token contracts.

RulesManagementModule



Events

event AddRule(address rule)

```
event AddRule(IRule indexed rule)
```

Emitted when a new rule is added to the rule set.

Event Parameters:

Name	Type	Description
rule	IRule	The address of the rule contract that was added.

event RemoveRule(address rule)

```
event RemoveRule(IRule indexed rule)
```

Emitted when a rule is removed from the rule set.

Event Parameters:

Name	Type	Description
rule	IRule	The address of the rule contract that was removed.

---

event ClearRules()

```
event ClearRules()
```

Emitted when all rules are cleared from the rule set.

This event has no parameters.

Functions

setRules(address[] rules_)

```
function setRules(IRule[] calldata rules_)
public virtual override(IRulesManagementModule)
onlyRole(RULES_MANAGEMENT_ROLE)
```

Defines the complete list of rules for the rule engine.

Any previously configured rules are completely replaced.
Rules must be deployed contracts implementing the expected `IRule` interface.
Reverts if any rule address is zero or if duplicates are detected.

This function calls _clearRules if at least one rule is still configured

Input Parameters:

Name	Type	Description
rules_	IRule[]	The array of IRule contracts to configure as the active rules.

---

rulesCount() -> uint256

```
function rulesCount()
public view virtual override(IRulesManagementModule)
returns (uint256)
```

Returns the total number of currently configured rules.

Equivalent to the length of the internal rules array.

Return Values:

Type	Description
uint256	The number of active rules.

---

rule(uint256 ruleId) -> address

```
function rule(uint256 ruleId)
public view virtual override(IRulesManagementModule)
returns (address)
```

Retrieves the rule address at a specific index.

Return the `zero address` if out of bounds.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Input Parameters:

Name	Type	Description
ruleId	uint256	The index of the desired rule in the array.

Return Values:

Type	Description
address	The address of the corresponding IRule contract.

---

rules() -> address[]

```
function rules()
public view virtual override(IRulesManagementModule)
returns (address[] memory)
```

Returns the full list of currently configured rules.

This is a view-only function and does not modify state.

This operation will copy the entire storage to memory, which can be quite expensive.

This is designed to mostly be used by view accessors that are queried without any gas fees.

Return Values:

Type	Description
address[]	An array containing all active rule contract addresses.

---

clearRules()

```
function clearRules()
public virtual override(IRulesManagementModule)
onlyRole(RULES_MANAGEMENT_ROLE)
```

Removes all configured rules.

After calling this function, no rules will remain set.

Developers should keep in mind that this function has an unbounded cost and using it may render the function uncallable if the set grows to the point where clearing it consumes too much gas to fit in a block.

---

addRule(address rule_)

```
function addRule(IRule rule_)
public virtual override(IRulesManagementModule)
onlyRole(RULES_MANAGEMENT_ROLE)
```

Adds a new rule to the current rule set.

Reverts if the rule address is zero or already exists in the set.

Input Parameters:

Name	Type	Description
rule_	IRule	The IRule contract to add.

---

removeRule(address rule_)

```
function removeRule(IRule rule_)
public virtual
override(IRulesManagementModule)
onlyRole(RULES_MANAGEMENT_ROLE)
```

Removes a specific rule from the current rule set.

Reverts if the provided rule is not found or does not match the stored rule at its index.

Input Parameters:

Name	Type	Description
rule_	IRule	The IRule contract to remove.

---

containsRule(address rule_) -> bool

```
function containsRule(IRule rule_)
public view virtual override(IRulesManagementModule)
returns (bool)
```

Checks whether a specific rule is currently configured.

Input Parameters:

Name	Type	Description
rule_	IRule	The IRule contract to check for membership.

Return Values:

Type	Description
bool	True if the rule is present, false otherwise.

Security

Vulnerability disclosure

Please see SECURITY.md (CMTAT main repository).

Audit

First Audit - March 2022

> The contracts (v.1.0.2) have been audited by ABDK Consulting, a globally recognized firm specialized in smart contracts' security.

Fixed version : v1.0.2

The first audit was performed by ABDK on the version 1.0.1.

The release v1.0.2 contains the different fixes and improvements related to this audit.

The final report is available in ABDK_CMTA_CMTATRuleEngine_v_1_0.pdf.

Tools

Nethermind AuditAgent

> **Note:** This scan was performed by an AI-powered automated tool, not a formal human-led audit.

Version	Report	Assessment
Scan #1 (Feb 2026)	audit_agent_report_1_v3.0.0-rc1.pdf	feedback.md

7 findings — 0 High · 1 Medium · 1 Low · 4 Info · 1 Best Practices

#	Severity	Finding	Status
1	Medium	Cross-token rule state pollution in multi-tenant deployments	NatSpec + README warnings. Interface fix deferred (requires CMTAT coordination).
2	Low	`RuleEngineOwnable` misreports `IAccessControl` via ERC-165	Fixed: explicit interface whitelist + negative test added.
3	Info	Unbounded rules loop — potential permanent DoS	Fixed in `v3.0.0-rc3`: on-chain configurable cap (`maxRules`) with default `10`, enforced in `addRule` and `setRules`.
4	Info	Restriction code and message can come from different rules	Convention documented in NatSpec and README (no logic change by design).
5	Info	Re-entrant rule can modify rule set during `transferred()`	Fixed in `v3.0.0-rc3`: rule accounts cannot receive roles in RBAC `RuleEngine`; ownable variants reject ownership transfer to rule accounts.
6	Info	Missing ERC-3643 and IERC7551Compliance interface IDs	Fixed: both IDs added to `supportsInterface` in both contracts, with tests.
7	Best Practices	`setRules` does not allow an empty array	NatSpec clarification added (behavior unchanged by design).

Slither

Here is the list of report performed with Slither

Version	Report	Assessment
v3.0.0-rc6	slither-report.md	slither-report-feedback.md

Version	Report	Assessment
v3.0.0-rc5	slither-report.md	slither-report-feedback.md
v3.0.0-rc4	slither-report.md	slither-report-feedback.md
v3.0.0-rc3	slither-report.md	slither-report-feedback.md
v3.0.0-rc2	slither-report.md	slither-report-feedback.md

```
slither .  --checklist --filter-paths "openzeppelin-contracts|test|CMTAT|forge-std|mocks" > slither-report.md
```

Latest run (v3.0.0-rc6, 108 contracts analysed, mocks excluded): 2 finding categories — 0 High · 0 Medium · 10 Low · 2 Informational. **Nothing to fix.**

ID	Detector	Impact	Instances	Assessment
0–9	`calls-loop`	Low	10	Accepted by design — t out to rule contracts is core architecture
10–11	`unindexed-event-address`	Informational	2	Accepted by design — `TokenBound` / `TokenUnl` keep the unindexed signature of the ERC-36 reference

Aderyn

Here is the list of report performed with Aderyn

```
aderyn -x mocks --output aderyn-report.md
```

Version	Report	Assessment
v3.0.0-rc6	aderyn-report.md	aderyn-report-feedback.md
v3.0.0-rc5	aderyn-report.md	aderyn-report-feedback.md
v3.0.0-rc4	aderyn-report.md	aderyn-report-feedback.md
v3.0.0-rc3	aderyn-report.md	aderyn-report-feedback.md
v3.0.0-rc2	aderyn-report.md	aderyn-report-feedback.md

Latest run (v3.0.0-rc6): 28 Solidity files, 683 nSLOC, mocks excluded.

0 High · 8 Low (84 instances). **Nothing to fix.**

ID	Finding	Instances	Assessment

ID	Finding	Instances	Assessment
L-1	Centralization Risk	14	Accepted by design — privileged compliance tool
L-2	Unspecific Solidity Pragma	23	Accepted by design — intentional for library reusability
L-3	PUSH0 Opcode	28	Not applicable — project targets Prague EVM
L-4	Modifier Invoked Only Once	1	Accepted by design — keeps hook-style access-control abstraction
L-5	Empty Block	9	Accepted by design — access-control hook pattern
L-6	Loop Contains `require` / `revert`	4	Accepted by design — `setRules` and `bindTokens` / `unbindTokens` are atomic batch operations
L-7	Costly Operations Inside Loop	4	Accepted — unavoidable `SSTORE` in batch operations
L-8	Unchecked Return	1	Accepted — `_grantRole` return is irrelevant in constructor

Overview of every analysis performed: AUDIT_OVERVIEW.md.

Documentation

Here a summary of the main documentation

Document	Link/Files
Integration with CMTAT	doc/technical/RuleEngine-with-CMTAT.md
Integration with ERC-3643	doc/technical/RuleEngine-with-ERC3643.md
Token binding module (reusable outside this project)	doc/technical/TokenBinding-module.md
Toolchain	doc/TOOLCHAIN.md
Surya report	doc/schema/surya
Code-quality review	doc/security/audits/tools/v3.0.0-rc6/CLAUDE_ANALYSIS.md
Script review	doc/security/audits/tools/v3.0.0-rc5/CLAUDE_ANALYSIS_SCRIPT.md
Audit overview	doc/security/audits/AUDIT_OVERVIEW.md

See also Taurus - Token Transfer Management: How to Apply Restrictions with CMTAT and ERC-1404 (RuleEngine v2.02 and CMTAT v2.4.0)

Toolchains and Usage

This repository is primarily developed and tested with Foundry.

Hardhat configuration is also present to allow compiling the contracts and running a small smoke test with Hardhat.

Parts of this project were written with the help of AI coding assistants, principally Claude Code (Anthropic) and Codex (OpenAI).

Configuration

Here are the settings for Hardhat and Foundry.

- `hardhat.config.js`
  - Solidity v0.8.36
  - EVM version: Prague (Pectra upgrade)
  - Optimizer: true, 200 runs
- `foundry.toml`
  - Solidity v0.8.36
  - EVM version: Prague (Pectra upgrade)
  - Optimizer: true, 200 runs

Toolchain installation

The contracts are developed and tested with Foundry, a smart contract development toolchain.

To install the Foundry suite, please refer to the official instructions in the Foundry book.

Initialization

You must first initialize the submodules, with

```
forge install
```

See also the command's documentation.

Later you can update all the submodules with:

```
forge update
```

See also the command's documentation.

Compilation

The official documentation is available in the Foundry website

```
# Build all contracts
forge build

# Build specific contract
forge build --contracts src/deployment/RuleEngine.sol
forge build --contracts src/deployment/RuleEngineOwnable.sol
forge build --contracts src/deployment/RuleEngineOwnable2Step.sol
```

Contract size

```
forge build --sizes
```

Latest output (`2026-03-18`) for the main RuleEngine contracts:

Contract	Runtime Size (B)	Initcode Size (B)	Runtime Margin (B)	Initcode Margin (B)
RuleEngine	6,756	7,805	17,820	41,347
RuleEngineOwnable	6,170	6,833	18,406	42,319

Both `RuleEngine` and `RuleEngineOwnable` remain well below the EIP-170 runtime limit. `RuleEngineOwnable` is slightly smaller because `Ownable` has less overhead than `AccessControl`.

Testing

You can run the tests with

```
forge test
```

To run a specific test, use

```
forge test --match-contract <contract name> --match-test <function name>
```

Generate gas report

```
forge test --gas-report
```

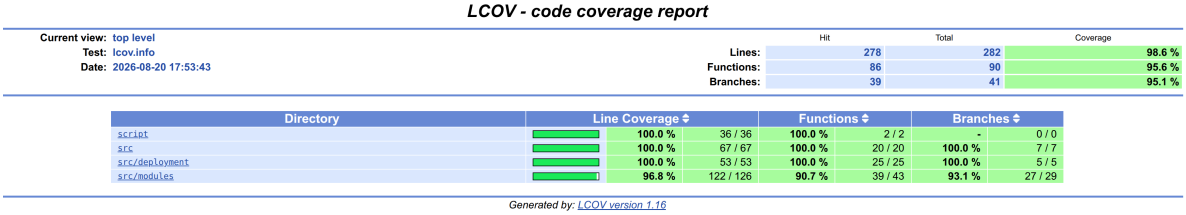
See also the test framework's official documentation, and that of the test commands.

There is also a small Hardhat smoke test to confirm the main `RuleEngine` contract can be compiled and deployed through Hardhat:

```
npx hardhat test test/hardhat/RuleEngine.smoke.js
```

Coverage

A code coverage is available in index.html.



LCOV - code coverage report

		Hit	Total	Coverage
Current view:	top level			
Test:	lcov.info	Lines:	278	282	98.6 %
Date:	2026-08-20 17:53:43	Functions:	86	90	95.6 %
		Branches:	39	41	95.1 %

Directory	Line Coverage		Functions		Branches	
script	100.0 %	36 / 36	100.0 %	2 / 2	-	0 / 0
src	100.0 %	67 / 67	100.0 %	20 / 20	100.0 %	7 / 7
src/deployment	100.0 %	53 / 53	100.0 %	25 / 25	100.0 %	5 / 5
src/modules	96.8 %	122 / 126	90.7 %	39 / 43	93.1 %	27 / 29

Generated by: LCOV version 1.16

- Perform a code coverage

```
forge coverage
```

- Generate LCOV report

```
forge coverage --report lcov
```

- Generate `index.html`

```
forge coverage --no-match-coverage "(mocks|test)" --report lcov && genhtml
lcov.info --branch-coverage --output-dir coverage
```

See Solidity Coverage in VS Code with Foundry & Foundry forge coverage

Deployment

The official documentation is available in the Foundry website

Choosing a Contract

Scenario	Recommended Contract
Multiple operators with different permissions	`RuleEngine`
Single administrator	`RuleEngineOwnable`
Single administrator with safer ownership handover	`RuleEngineOwnable2Step`
Integration with existing RBAC systems	`RuleEngine`
Simpler deployment and management	`RuleEngineOwnable`

Script

The scripts in `script/` are example deployment flows.

> Warning: `RuleEngineScript.s.sol` and `CMTATWithRuleEngineScript.s.sol` deploy `RuleWhitelistMock` from `src/mocks/`. That contract is a reference/mock rule for testing and demos, not a production rule contract.

For production deployments, source rule contracts from the dedicated CMTA/Rules repository and adapt the script parameters accordingly.

To run the example scripts, create a `.env` file. The value for `CMTAT_ADDRESS` is required only for `RuleEngineScript.s.sol`.

Warning: putting your private key in a .env file is not the most secure approach.

- File `.env`

```
PRIVATE_KEY=<YOUR_PRIVATE_KEY>
CMTAT_ADDRESS=<CMTAT_ADDRESS>
```

Private Keys: Never expose your private keys. The `.env` file here used in this project should not be used for production. See getfoundry.sh - Key Management

- Command

CMTAT with RuleEngine

```
forge script script/CMTATWithRuleEngineScript.s.sol:CMTATWithRuleEngineScript --rpc-url=$RPC_URL  --broadcast --verify -vvv
```

- Value of YOUR_RPC_URL with a local instance of anvil : 127.0.0.1:8545

```
forge script script/CMTATWithRuleEngineScript.s.sol:CMTATWithRuleEngineScript --rpc-url=127.0.0.1:8545  --broadcast --verify -vvv
```

Only RuleEngine with the mock/reference `RuleWhitelistMock` contract

```
forge script script/RuleEngineScript.s.sol:RuleEngineScript --rpc-url=$RPC_URL
 --broadcast --verify -vvv
```

- With anvil

```
forge script script/RuleEngineScript.s.sol:RuleEngineScript --rpc-url=127.0.0.1:8545  --broadcast --verify -vvv
```

Production Deployment Checklist

- Choose the deployable variant: `RuleEngine`, `RuleEngineOwnable`, or `RuleEngineOwnable2Step`.
- Choose the trusted forwarder address, or use `address(0)` if ERC-2771 support is not needed.
- Decide whether the token should be bound in the constructor or later via `bindToken`.
- Source production rule contracts from the CMTA/Rules repository, not from `src/mocks/`.
- Verify post-deployment permissions: owner for ownable variants, or admin plus role assignments for the RBAC variant.

Solidity style guideline

RuleEngine follows the solidity style guideline and the natspec format for comments

Formatting & Linting

We use Foundry's built-in formatter and linter:

```
# Format all Solidity files
forge fmt

# Check formatting without modifying files
forge fmt --check

# Run the Solidity linter
forge lint
```

- Orders of Functions

Functions are grouped according to their visibility and ordered:

```
1. constructor

2. receive function (if exists)

3. fallback function (if exists)

4. external

5. public

6. internal

7. private
```

Within a grouping, place the `view` and `pure` functions last

- Function declaration

```
1. Visibility
2. Mutability
3. Virtual
4. Override
5. Custom modifiers
```

Intellectual property

The code is copyright (c) Capital Market and Technology Association, 2022-2026, and is released under Mozilla Public License 2.0.